



SIMULACIONES Y PROCESOS ESTOCÁSTICOS

2026

Teoría: Dr. Alejandro Mezio – mezioalejandro@uca.edu.ar

Práctica: Dr. Luis Herrera – lherrera@uca.edu.ar



PROCESOS DE DECISIÓN DE MARKOV Y ECUACIÓN DE BELLMAN

- G. Ciaburro, Hand-os Simulations...: Capítulo 5

Motivación: ya vimos

- Sistemas que evolucionan con el tiempo
- Procesos estocásticos
- Cadenas de Markov

Son sistemas que evolucionan “solos”

¿Pero qué pasa si podemos tomar decisiones?

¿Qué sucede cuando podemos intervenir y decidir?

Repaso:

Cadenas de Markov

- Estados s
- Transiciones $P(s'|s)$ (*acá reservamos la notación Q para otra cantidad*)
- Propiedad de Markov: probabilidad de que ocurra un evento depende solamente del evento previo

El futuro depende sólo del presente

Cadenas de Markov → evolución pasiva

Procesos de decisión de Markov → decisiones óptimas

Nuevas preguntas:

- ¿Qué decisión tomar en cada estado?
- ¿Cómo optimizar el comportamiento del sistema?

Los Procesos de Decisión de Markov (MDP) agregan:
las acciones a y las recompensas $R(s, a)$

Las 5 componentes de un MDP son:

- **Conjunto de estados S** ($s, s' \in S$)
Igual que antes: posibles estados del sistema
- **Conjunto de acciones A** ($a \in A$)
Lo nuevo: las transiciones ya no son automáticas, podemos elegir una acción
- **Transiciones $P(s'|s, a)$**
Probabilidad de ir al estado s' desde s , haciendo la acción a
- **Recompensas $R(s, a)$**
Qué ganás (o perdés) al hacer la acción a estando en el estado s
- **Factor de descuento γ**
Cuánto te importan las recompensas futuras ($\gamma \in [0,1]$)



Cadena de Markov	MDP
Evolución pasiva	Decisiones
Sin control	Control del sistema

Objetivo de un MDP:

Encontrar una política $\pi(s)$ que maximice la recompensa acumulada

Función de valor:

(estar) • $V(s)$: valor del estado s

(hacer) • $Q(s, a)$: valor de tomar la acción a estando en s

¿Qué pasa si tomo la acción a en este estado s ?

Idea de la ecuación de Bellman:

VALOR = RECOMPENSA INMEDIATA + VALOR FUTURO

Principio fundamental de la optimización dinámica

Equilibrio entre presente y futuro

Función de valor:

$$V(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right]$$

- Se evalúan todas las acciones
- Se consideran todos los posibles futuros
- Se elige la mejor opción

Se elige hoy teniendo en cuenta cómo eso afecta todo el futuro

Función de valor:
$$V(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right]$$

- El término $\sum_{s'} P(s'|s, a) V(s')$ es un promedio ponderado de los futuros posibles
- Pero podemos pasar de un caso estocástico a uno determinista si la cantidad $P(s'|s, a)$ toma el valor 1 para un determinado s' y 0 para el resto
En este caso no hay incertidumbre, para cada acción hay un único estado siguiente posible.

Función de valor determinista:
$$V(s) = \max_a [R(s, a) + \gamma V(s')]$$

Observemos que, en ambos casos, para cada estado s del sistema encontramos la acción a adecuada que maximiza el valor. **Encontramos la política óptima $\pi(s)$**

$\pi(s)$ = acción a tomar en el estado s

Factor de descuento γ

Controla cuánto te importa el futuro respecto al presente

- $\gamma = 0$: sólo te importa el presente
- $\gamma = 1$: te importa mucho el futuro

Ejemplo: tengo dos opciones

- Opción A: ganar 10 ahora
- Opción B: ganar 1 por paso para siempre



- γ chico: preferís A
- γ grande: preferís B

$$V(s) = \max_a \left[R(s, a) + \underbrace{\gamma \sum_{s'} P(s'|s, a) V(s')}_{\text{futuro}} \right]$$

El futuro se “achica” por γ

1. Penaliza el futuro (más lejos, menos pesa)
La recompensa t iteraciones mas tarde vale γ^t
2. Evita infinitos
Sumar infinitas recompensas puede diverger
3. Define el horizonte efectivo
Sólo importa corto/mediano plazo
 $\gamma \approx 0.9 \rightarrow$ horizonte aprox 10 pasos
 $\gamma \approx 0.99 \rightarrow$ horizonte aprox 100 pasos

Conexión:

- Una Cadena de Markov es un MDP sin decisiones
- Si fijamos una política $\pi \rightarrow$ entonces el MDP se vuelve una Cadena de Markov

Los MDP son la base de:

- Aprendizaje por refuerzo
- Robótica
- Finanzas (decisiones secuenciales)
- Sistemas de recomendación

Ejemplo sencillo: MDP con 2 estados

- Estados: $S = \{A, B\}$
- Acciones: $A = \{a_1, a_2\}$
- Dinámica del sistema:

Desde A:

- a_1
 - Va a A con probabilidad 1
 - Recompensa = 2
- a_2
 - Va a B con probabilidad 1
 - Recompensa = 0

Desde B:

- a_1
 - Va a A con probabilidad 1
 - Recompensa = 1
- a_2
 - Va a B con probabilidad 1
 - Recompensa = 3

Nuestro objetivo es encontrar la política óptima π^*

Ejemplo sencillo: MDP con 2 estados

Notemos que estamos en un caso determinista: $V(s) = \max_a [R(s, a) + \gamma V(s')]]$

$$V(A) = \max \left\{ \underbrace{2 + 0.9V(A)}_{a_1}, \underbrace{0 + 0.9V(B)}_{a_2} \right\}$$

$$V(B) = \max \left\{ \underbrace{1 + 0.9V(A)}_{a_1}, \underbrace{3 + 0.9V(B)}_{a_2} \right\}$$

¿Cómo hacemos? Probemos qué acción conviene en cada estado

Desde A:

- a_1 : $2 + 0.9 V(A)$
- a_2 : $0 + 0.9 V(B)$

- a_1 da mayor recompensa inmediata (2 vs 0), pero ir a B puede resultar mejor ya que estando ahí se puede ganar 3
- Intuición: parece haber un trade-off

Desde B:

- a_1 : $1 + 0.9 V(A)$
- a_2 : $3 + 0.9 V(B)$

- a_2 da mayor recompensa inmediata (3 vs 1), y además nos deja en B (donde podemos seguir ganando)
- Intuición: en B conviene a_2

Ejemplo sencillo: MDP con 2 estados

Intentemos con una política óptima

1

- En $A \rightarrow a_1$
- En $B \rightarrow a_2$



Para A:

$$V(A) = 2 + 0.9V(A)$$

$$0.1 V(A) = 2$$

$$V(A) = 20$$

Para B:

$$V(B) = 3 + 0.9V(B)$$

$$0.1 V(B) = 3$$

$$V(B) = 30$$

Chequeamos entonces si esta política resulta óptima
(es decir, si son las acciones que maximizan el valor de cada estado)

Desde A:

- $a_1: 2 + 0.9 V(A) = 20$
- $a_2: 0 + 0.9 V(B) = 27$

a_2 es mejor!

Nuestra política no es óptima

Desde B:

- $a_1: 1 + 0.9 V(A) = 19$
- $a_2: 3 + 0.9 V(B) = 30$

a_2 sería la correcta aquí

Ejemplo sencillo: MDP con 2 estados

Intentemos con OTRA política óptima

2

- En $A \rightarrow a_2$
- En $B \rightarrow a_2$



Para A:

$$V(A) = 0 + 0.9V(B)$$

$$V(A) = 0.9 \times 30$$

$$V(A) = 27$$

Para B:

$$V(B) = 3 + 0.9V(B)$$

$$0.1 V(B) = 3$$

$$V(B) = 30$$

Chequeamos:

Desde A:

- $a_1: 2 + 0.9 V(A) = 26.3$
- $a_2: 0 + 0.9 V(B) = 27$

Desde B:

- $a_1: 1 + 0.9 V(A) = 25.3$
- $a_2: 3 + 0.9 V(B) = 30$



Política óptima:

En $A \rightarrow$ ir a B (a_2)

En $B \rightarrow$ quedarse en B (a_2)

El estado B es “rico” (da siempre 3).
Desde A conviene ir a B lo antes posible, y
quedarse ahí explotando la recompensa

Métodos para usar la ecuación de Bellman

- Policy evaluation: fijo una política π y calculo qué tan buena es.
Sirve para medir cuánto voy a ganar si sigo una cierta política.
Evaluamos Bellman sin $\max_a$
- Value iteration: lo que usamos en el ejemplo de recién.
Sin mantener una política explícita, voy actualizando los valores V
- Policy iteration: similar, pero resolvemos la ecuación de Bellman para $Q(s, a)$

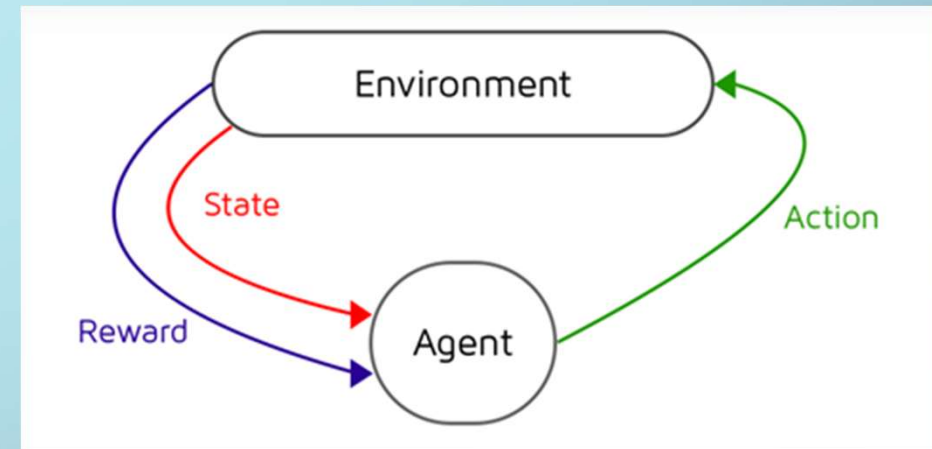
Esto es Programación Dinámica: resolver problemas complejos dividiéndolos en subproblemas óptimos

Lo que nos permite resolver los MDP usando Programación Dinámica es el **principio de optimalidad**: Una política óptima tiene la propiedad de que, desde cualquier estado, las decisiones futuras también son óptimas

Modelos basados en agentes

¿Qué es un Agent-Based Model?

- Sistema compuesto por múltiples agentes
- Cada agente tiene:
 - Estado
 - Reglas de comportamiento
- Interactúan entre si y con el entorno



De esta manera, el comportamiento global emerge de interacciones locales

Idea central: en vez de modelar ecuaciones agregadas (agrupadas), ahora modelamos individuos que toman decisiones

Modelos basados en agentes

Considerando las interacciones de los elementos individuales del sistema, tratamos de determinar las características que emergen de estas interacciones

- Proceso de Decisión de Markov (MDP) → modela decisiones óptimas de 1 agente
- Modelo Basado en Agentes (ABM) → modela interacción de muchos agentes

Dos tipos de agentes:

REACTIVOS

- Siguen reglas simples
- No optimizan

RACIONALES

- Toman decisiones
- Optimizan comportamiento

Modelos basados en agentes

Cada agente:

- Observa el estado del sistema s
- Elige acción a
- Recibe recompensa

Estructura de un MDP

Modelos basados en agentes

Ejemplo: tráfico urbano

- Agentes: conductores
- Estados: posición, velocidad
- Acciones: acelerar, frenar

Cada gente puede optimizar:

- Tiempo de viaje
- Consumo

Modelos basados en agentes

Si el agente no conoce el entorno: aprende usando **aprendizaje por refuerzo (RL)**

- Observa
- Actúa
- Aprende

MDP + ABM permite modelar sistemas donde hay:

- Múltiples decisiones
- Interacción
- Aprendizaje

Resumen

Enfoque	Qué modela
Cadenas de Markov	Dinámica sin decisiones
MDP	Decisión óptima
ABM	Interacción entre agentes
RL	Aprendizaje de decisiones

A decorative graphic on the left side of the slide, consisting of a network of blue lines and circles resembling a circuit board or a neural network. The lines are of varying thickness and connect to small circles at various points.

Ejemplo modelo de Schelling → repositorio Ciaburro

Concepto importante: ecuaciones de Punto Fijo

$x = f(x)$ se nos aparece constantemente (aunque en formas más complejas)

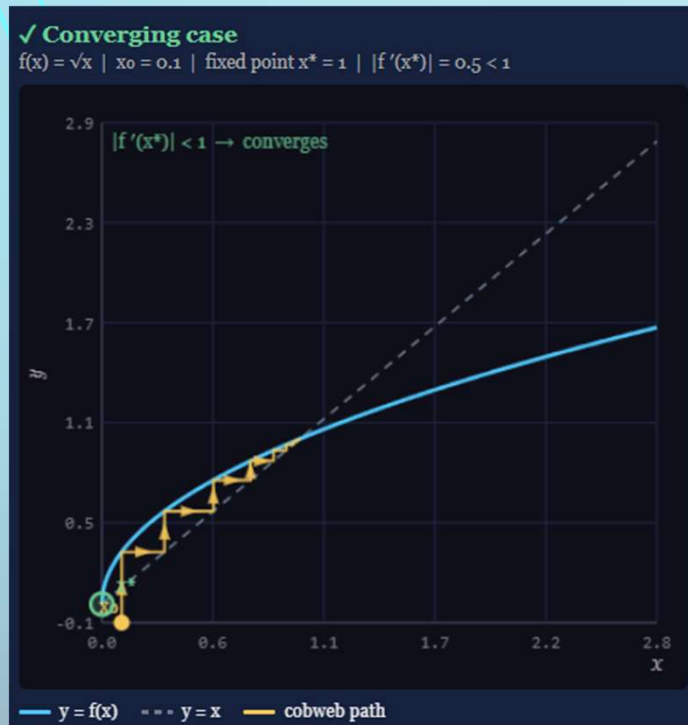
Método más usado: el iterativo

$$x_0 \rightarrow x_1 = f(x_0) \rightarrow x_2 = f(x_1) \rightarrow \cdots \rightarrow x_{n+1} = f(x_n)$$

Si en algún momento $x_{n+1} \approx x_n$, entonces convergió a una solución

- Existen teoremas y condiciones que puede cumplir $f(x)$ que nos GARANTIZAN la convergencia.
- Esta familia de $f(x)$ se las llaman “contracciones”

Concepto importante: ecuaciones de Punto Fijo



La variable x puede ser un vector de muchas dimensiones, y la función $f(x)$ ser muy compleja (como la Ecuación de Bellman!)



IDEA A LLEVARSE: parece todo complejo, pero no es mas que $x = f(x)$ más elaborado. Lo importante es garantizarnos $f(x)$ contracción

Concepto importante: ecuaciones de Punto Fijo

Variante muy usada en Machine Learning: hay una “tasa de aprendizaje” α , y el nuevo x_{n+1} es un poco del valor viejo x_n y un poco del valor nuevo $f(x_n)$

- Elijo un x_0
- $x_1 = (1 - \alpha) x_0 + \alpha f(x_0)$
- $x_2 = (1 - \alpha) x_1 + \alpha f(x_1)$
- ...
- $x_{n+1} = (1 - \alpha) x_n + \alpha f(x_n)$

$$\alpha \in [0,1]$$

Learning rate α : ayuda a converger, suavizando los cambios, evitando saltos abruptos que nos puedan llevar a valores lejanos

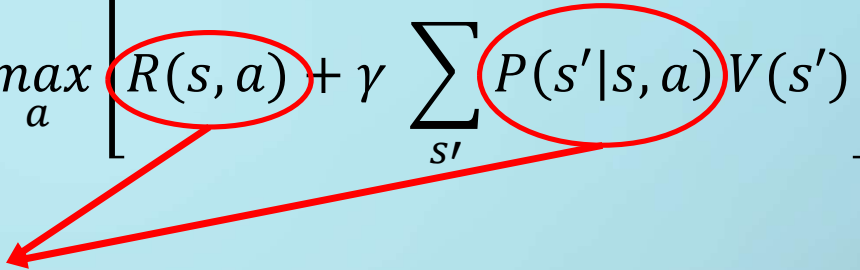
Notar que podemos escribir
$$x_{n+1} = x_n + \alpha(f(x_n) - x_n)$$

“error”, lo
que $f(x_n)$
corrige a x_n

$f(x_n)$ puede llamarse “target”

Q-learning y aprendizaje por refuerzo

Ecuación de Bellman:
$$V(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right]$$



Antes, vimos casos donde conocíamos $R(s, a)$ y $P(s'|s, a)$

¿Qué sucede cuando no conocemos P ?

¿Y si R no lo tenemos de antemano?

Tenemos que ir **probando acciones y observando consecuencias**

- **Model-based Reinforcement Learning:** Obtenemos estimaciones de $R(s, a)$ y $P(s'|s, a)$ a partir de la experiencia, y luego resolvemos el MDP asociado.
- **Model-free Reinforcement Learning:** Aprendemos directamente la política óptima o la función valor desde la experiencia, no estimamos $P(s'|s, a)$
 - **Q-learning:** estimar $Q(s, a)$ directamente (vamos a ver esto)

Q-learning y aprendizaje por refuerzo

Supongamos que existe una política óptima π^* , la función de valor óptima $V^*(s)$ satisface la ecuación de Bellman:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right]$$

Recordemos $Q(s, a)$, la función de valor de la acción a en el estado s nombrada antes. La relación de la función de valor óptima $V^*(s)$ con la función Q -valor óptima $Q^*(s, a)$ es:

$$V^*(s) = \max_a [Q^*(s, a)]$$

Entonces la ecuación de Bellman para la función Q -valor óptima es:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} [Q^*(s', a')]]$$

De esta manera, la política óptima resulta: $\pi^*(s) = \arg \max_a [Q^*(s, a)]$

En s tomo la acción a que maximiza el valor de Q

Q-learning y aprendizaje por refuerzo

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} [Q^*(s', a')]]$$

- Recordemos que ahora estamos en la situación que no conocemos $P(s'|s, a)$
- Pero podemos observar transiciones. Tendremos como datos: s, a, R, s'
- Cada una de estas transiciones nos darán un “target”, un Q nuevo:

$$\text{target}(s, a) = R + \gamma \max_{a'} [Q(s', a')]]$$

- Entonces, Q -learning es actualizar con un cierto learning rate α :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} [Q(s', a')]] - Q(s, a) \right]$$

Recordemos: $x_{n+1} \leftarrow x_n + \alpha [f(x_n) - x_n]$

Q-learning y aprendizaje por refuerzo

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{R + \gamma \max_{a'} [Q(s', a')]}_{\text{Target, o nueva evidencia}} - \underbrace{Q(s, a)}_{\text{Valor viejo}} \right]$$

Target, o nueva
evidencia

Valor viejo

Error de Bellman (TD error)

$\alpha \in [0,1]$. Cuánto peso le damos a la nueva evidencia

- Con suficiente exploración y α que va decreciendo se garantiza la convergencia $Q \rightarrow Q^*$

Q-learning y aprendizaje por refuerzo

El dilema exploración-explotación

El agente se encuentra en cada decisión con un dilema:

- Usar lo que ya sabe (explotar lo conocido)
- Probar un camino nuevo que podrían ser mejor (explorar lo desconocido)

EXPLOTACIÓN: Elegir la mejor acción conocida. Esto maximiza la recompensa inmediata

EXPLORACIÓN: Elegir acciones menos conocidas. Pueden descubrirse mejores opciones de recompensa futura

Supongan estamos eligiendo restaurante:

- Elijo siempre uno bueno que conozco (explotar) → me quedo con algo bueno pero no óptimo
- Elijo siempre probar alguno nuevo (explorar) → puedo conseguir algo más óptimo, pero si exploro demasiado puedo perder tiempo y/o calidad. No aprovecho lo aprendido.

No podemos hacer ambas cosas al mismo tiempo → HAY UN TRADE-OFF

Q-learning y aprendizaje por refuerzo

El dilema exploración-explotación

Estrategia común: Política ε -greedy

$$\pi_{\varepsilon}(s) = \begin{cases} \text{acción aleatoria} & \text{con probabilidad } \varepsilon \\ \arg \max_a [Q(s, a)] & \text{con probabilidad } 1-\varepsilon \end{cases}$$

La cantidad ε decrece con el número de episodios:

- Al inicio el agente conoce poco, entonces queremos ε grande
- Luego de mucha exploración, al tener más conocimiento sobre el sistema, queremos que $Q \approx Q^*$ y necesitamos ε pequeño

Esquema típico: $\varepsilon_t = \varepsilon_{\min} + (\varepsilon_{\max} - \varepsilon_{\min})^{-t/\tau}$

Q-learning y aprendizaje por refuerzo

MDP sin $P(s'|s, a) \rightarrow$ Reinforcement Learning (agente aprende $Q^*(s, a)$ interactuando con el entorno)

Q-learning es un caso particular, donde Q se actualiza hacia el target con cada transición (s, a, R, s')

Ejemplo conceptual: tráfico en una intersección

Entorno:

- Intersección simple, tipo cruce
- Autos llegan desde ambas direcciones

Cada auto es un agente, con:

- Estado
 - Posición
 - Velocidad
 - Distancia hasta el cruce
- Acciones
 - Mantener velocidad
 - Acelerar
 - Frenar

Dinámica básica:

- Pasos de tiempo discretos
- Posibilidad de colisión
- Congestión si hay muchos autos

Ejemplo conceptual: tráfico en una intersección

Caso 1: Agentes con reglas fijas (ABM puro)

- Si hay auto adelante → frenar
- Si no hay auto adelante → avanzar
- Prioridad: llegar a cruzar

¿Qué resultado esperamos?

- Comportamiento razonable
- Pero cada auto sólo piensa en lo suyo → congestión
- Ineficiencias

El comportamiento emerge
sin optimización

Ejemplo conceptual: tráfico en una intersección

Caso 2: Agentes como MDP (con optimización)

Ahora cada agente decide si acelerar o frenar para optimizar “algo”

Estados s del MDP:

- Distancia al cruce
- Si hay auto adelante o no
- Estado del cruce

Acciones a del MDP:

- Mantener
- Acelerar
- Frenar

Hay optimización individual

Recompensa $R(s, a)$ del MDP:

- +1 avanzar
- -10 colisión
- -1 esperar

Objetivo (el “algo”): Minimizar tiempo sin chocar

Resultado esperado:

- Decisiones más eficientes
- Mejor tráfico

Ejemplo conceptual: tráfico en una intersección

Caso 3: Agentes con Q-learning

Ahora cada agente no conoce el modelo y aprende con la experiencia

Dinámica de cada agente:

- Observa el estado
- Elige la acción
- Recibe recompensa
- Actualiza Q

Qué podemos usar como métrica:

- Tiempo promedio de cruce
- Número de colisiones
- Nivel de congestión

Qué se **observa** en la simulación:

- Al comienzo:
 - Comportamiento caótico
- Luego:
 - Aprendizaje gradual
- Finalmente:
 - Flujo eficiente



FIN